CELL 6 Flow and Optimization Guide

Related Documents

- **CELL6_OPTIMIZATION_RECOMMENDATIONS.md**: Quick-start guide with recommended settings
- **PIPELINE_CONFIG_OVERVIEW.md**: Full description of `CELL6_OPTIMIZE`, per-cell params, and merge behavior

---

Executive Summary

This document explains how CELL 6A, 6B, 6C, and 6D work together, how `CELL6_OPTIMIZE` controls them, and the data flow between cells.

Quick Start: If you just want recommended settings for a specific scenario, see CELL6_OPTIMIZATION_RECOMMENDATIONS.md.

Core Concepts

CELL 6 is a four-stage, resumable pipeline. Each stage builds on the last, and configuration flows from global defaults to cell-specific overrides. This structure is designed to avoid recomputation while keeping the pipeline configurable.

Assignment modes: TB metrics and pool membership logic are config-driven in `pipeline_config.py`.
- `TB_ASSIGNMENT_MODE_SELF` controls forward vs backward TB assignment for self metrics
- `TB_ASSIGNMENT_MODE_POOL` controls forward vs backward TB assignment for pool construction
- `TB_PRECOMPUTE_BOTH_MODES` optionally stores both modes once (`*_fwd`, `*_bwd`)
- Pool assignment always uses overlap logic (start_date ≤ snapshot ≤ thru_date)
- `OER_NAT_POLICY` controls how missing OER dates are handled

Overview: Four-Cell Structure

CELL 6 has been split into four resumable cells:

Cell	Purpose	Output File	Typical Runtime	Required?
CELL 6A	Basic cumulative metrics	`df_pipeline_03a_basic_metrics.feather`	~30 minutes	✅ Always required
CELL 6B	Optional metrics (ratings given, grade, last4)	`df_pipeline_03b_optional_metrics.feather`	~30-60 minutes	❌ Can skip
CELL 6C	Pre-pool prep (no pool metrics)	`df_pipeline_03c_pre_pool_metrics.feather`	~minutes to ~1 hour	❌ Can skip
CELL 6D	Pool metrics (comparative within pools)	`df_pipeline_03_oer_metrics.feather`	~1-2 hours	❌ Can skip

Key Design: Each cell is **resumable**—it checks for its output file and skips computation if found.

CELL6_OPTIMIZE Configuration

Location

`CELL6_OPTIMIZE` is defined in `pipeline_config.py` as a dictionary:

```
CELL6_OPTIMIZE = {
    # Data Quality Settings (used in CELL 6A)
    'check_data_quality': False,      # Set False if data already cleaned (saves 1-2 hours)
    'report_duplicates': False,        # Set False to skip duplicate reporting
    'report_overlaps': False,          # Set False to skip overlap reporting
    'clean_duplicates': True,         # Keep True to remove duplicates (fast operation)

    # Optional Metrics Settings (used in CELL 6B)
    'include_ratings_given': True,     # Set False if not needed (saves ~30 min)
    'include_grade_metrics': True,     # Set False if not needed (saves ~15 min)
    'include_last4_metrics': True,    # Set False if not needed (saves ~15 min)

    # Pool Metrics Settings (used in CELL 6D)
    'include_pool_metrics': True,     # Set False if not needed (saves 1-2 hours)
    'use_optimized_pool_metrics': True,  # Keep True (optimized pool metrics path)

    # Performance Profiling (used in all cells)
    'profile_performance': True,       # Set True to profile which parts take longest
}
```

How CELL 6 gets parameters now

CELL 6 uses `get_cell6_params()` (defined in `pipeline_config.py`) to merge: - `CELL6_OPTIMIZE` (global defaults) - Cell-specific params (`CELL6A_PARAMS`, `CELL6B_PARAMS`, `CELL6C_PARAMS`, `CELL6D_PARAMS`)

Any value set to `None` in a cell-specific params dict inherits from `CELL6_OPTIMIZE`. Merge order (later wins): `CELL6_COLUMN_MAPPING` → `CELL6_ADDITIONAL_PARAMS` → `CELL6_OPTIMIZE` → `CELL6A/B/C_PARAMS`.

How Each Cell Uses CELL6_OPTIMIZE

CELL 6A: Basic Metrics

Reads from CELL6_OPTIMIZE: - `check_data_quality` → Controls data quality checks - `report_duplicates` → Controls duplicate reporting - `report_overlaps` → Controls overlap reporting - `clean_duplicates` → Controls duplicate cleaning - `profile_performance` → Controls performance profiling

Hardcoded Settings (not from CELL6_OPTIMIZE): - `include_ratings_given=False` (always False in 6A) - `include_grade_metrics=False` (always False in 6A) - `include_last4_metrics=False` (always False in 6A) - `include_pool_metrics=False` (always False in 6A)

Code:

```
df_with_metrics = add_cumulative_oer_metrics(
    # ... other params ...
    check_data_quality=CELL6_OPTIMIZE.get('check_data_quality', True),
    report_duplicates=CELL6_OPTIMIZE.get('report_duplicates', True),
    report_overlaps=CELL6_OPTIMIZE.get('report_overlaps', True),
    clean_duplicates=CELL6_OPTIMIZE.get('clean_duplicates', True),
    include_ratings_given=False,   # Hardcoded - not from CELL6_OPTIMIZE
    include_grade_metrics=False,   # Hardcoded - not from CELL6_OPTIMIZE
    include_last4_metrics=False,  # Hardcoded - not from CELL6_OPTIMIZE
    include_pool_metrics=False,   # Hardcoded - not from CELL6_OPTIMIZE
)
```

CELL 6B: Optional Metrics

Reads from CELL6_OPTIMIZE: - `include_ratings_given` → Controls ratings given metrics - `include_grade_metrics` → Controls grade-based metrics - `include_last4_metrics` → Controls last4 (rolling window) metrics

Hardcoded Settings (not from CELL6_OPTIMIZE): - `check_data_quality=False` (skipped - already done in 6A) - `report_duplicates=False` (skipped - already done in 6A) - `report_overlaps=False` (skipped - already done in 6A) - `clean_duplicates=False` (skipped - already done in 6A) - `include_pool_metrics=False` (always False in 6B)

Code:

```
include_ratings = CELL6_OPTIMIZE.get('include_ratings_given', True)
include_grade = CELL6_OPTIMIZE.get('include_grade_metrics', True)
include_last4 = CELL6_OPTIMIZE.get('include_last4_metrics', True)

df_with_optional = add_cumulative_oer_metrics(
    # ... other params ...
    include_ratings_given=include_ratings,  # From CELL6_OPTIMIZE
    include_grade_metrics=include_grade,    # From CELL6_OPTIMIZE
    include_last4_metrics=include_last4,   # From CELL6_OPTIMIZE
    check_data_quality=False,  # Hardcoded - skip (already done in 6A)
    include_pool_metrics=False, # Hardcoded - not from CELL6_OPTIMIZE
)
```

CELL 6C: Pre-Pool Prep (no pool metrics)

Reads from CELL6_OPTIMIZE: - `include_ratings_given`, `include_grade_metrics`, `include_last4_metrics` (fallback only)

Hardcoded Settings (not from CELL6_OPTIMIZE): - `include_pool_metrics=False` (pre-pool only) - `skip_cum_recompute=True` (avoid recomputing if present) - `check_data_quality=False` (skipped - already done in 6A) - `report_duplicates=False`, `report_overlaps=False`,

`clean_duplicates=False`

Optional Postgres push: - If `POSTGRES_POOL_PUSH_6C=True`, CELL 6C pushes `df_pipeline_03c_pre_pool_metrics` to `POSTGRES_POOL_INPUT_TABLE` using `POSTGRES_POOL_PG_URL`. - If `POSTGRES_POOL_RUN_SQL=True`, CELL 6C runs the SQL to create `POSTGRES_POOL_OUTPUT_TABLE`.

Code:

```
df_pre_pool = add_cumulative_oer_metrics(
    # ... other params ...
    include_pool_metrics=False,   # Pre-pool only
)
```

CELL 6D: Pool Metrics

Reads from CELL6_OPTIMIZE: - `include_pool_metrics` → Controls pool metrics - `use_optimized_pool_metrics` → Optimized pool path

Optional Postgres path: - If `USE_POSTGRES_POOL_METRICS=True`, CELL 6D reads from Postgres instead of computing in pandas. See `current_documents/README_postgres_pool_metrics.md` for the checklist.

Hardcoded Settings (not from CELL6_OPTIMIZE): - `skip_cum_recompute=True` (avoid recomputing if present) - `check_data_quality=False`, `report_duplicates=False`, `report_overlaps=False`, `clean_duplicates=False`

Code:

```
df_with_pool = add_cumulative_oer_metrics(
    # ... other params ...
    include_pool_metrics=True,  # Pool metrics only
)
```

Summary: What CELL6_OPTIMIZE Controls

Setting	Used In	Controls
`check_data_quality`	CELL 6A only	Data quality checks (expensive)
`report_duplicates`	CELL 6A only	Duplicate reporting
`report_overlaps`	CELL 6A only	Overlap reporting
`clean_duplicates`	CELL 6A only	Duplicate cleaning (fast)
`include_ratings_given`	CELL 6B (fallback in 6C/6D if needed)	Ratings given metrics
`include_grade_metrics`	CELL 6B (fallback in 6C/6D if needed)	Grade-based metrics
`include_last4_metrics`	CELL 6B (fallback in 6C/6D if needed)	Last4 (rolling window) metrics
`include_pool_metrics`	CELL 6D	Pool metrics
`use_optimized_pool_metrics`	CELL 6D	Use optimized pool metrics path
`profile_performance`	All cells	Performance profiling

Key Insight: `CELL6_OPTIMIZE` provides **defaults**, but each cell has **hardcoded overrides** for certain settings to optimize the flow.

Data Flow Between Cells

Flow Diagram

```
CELL 5 Output: df_pipeline_02_oer_joined.feather
    |
    ↓
[CELL 6A] add_cumulative_oer_metrics()
    |
    ├─ Computes: Basic cumulative metrics
    |  • cum_tb_rcvd_rtr, cum_tb_rcvd_snr
    |  • cum_evals_rcvd_rtr, cum_evals_rcvd_snr
    |  • tb_ratio_rtr, tb_ratio_snr
    |
    ├─ Data Quality: check_data_quality, clean_duplicates (if enabled)
    |
    └─ Saves: df_pipeline_03a_basic_metrics.feather
        |
        ↓
[CELL 6B] add_cumulative_oer_metrics()
    |
    ├─ Loads: df_pipeline_03a_basic_metrics.feather (from 6A)
    |
    ├─ Checks: Are basic metrics present? (Yes, from 6A)
    |
    ├─ Computes: Basic metrics AGAIN + Optional metrics
    |  • Recomputes: cum_tb_rcvd_rtr, cum_tb_rcvd_snr, etc. (redundant)
    |  • Adds: cum_tb_given_rtr, cum_tb_given_snr (if enabled)
    |  • Adds: co_grade_tb_rtr, field_grade_tb_rtr (if enabled)
    |  • Adds: last4_tb_rtr, last4_tb_snr (if enabled)
    |
    ├─ Data Quality: SKIPPED (already done in 6A)
    |
    └─ Saves: df_pipeline_03b_optional_metrics.feather
        |
        ↓
[CELL 6C] add_cumulative_oer_metrics()
    |
    ├─ Loads: df_pipeline_03b_optional_metrics.feather (from 6B)
    |         OR df_pipeline_03a_basic_metrics.feather (from 6A, if 6B skipped)
    |
    ├─ Pre-pool prep (no pool metrics)
    |  • Optional: precompute both TB modes (`*_fwd`, `*_bwd`)
    |
    ├─ Data Quality: SKIPPED (already done in 6A)
    |
    └─ Saves: df_pipeline_03c_pre_pool_metrics.feather
        |
        ↓
[CELL 6D] add_cumulative_oer_metrics()
    |
    ├─ Loads: df_pipeline_03c_pre_pool_metrics.feather
    |
    ├─ Computes: Pool metrics only
    |  • Adds: pool_cum_tb_mean_rtr, cum_tb_diff_rtr, etc.
    |
    └─ Saves: df_pipeline_03_oer_metrics.feather (FINAL OUTPUT)
```

Key Observations

1. **Each cell calls `add_cumulative_oer_metrics()` from scratch**
    - The function doesn't have an "add-only" mode
    - It recomputes everything it's asked to compute

2. **Redundancy exists**:
   - CELL 6B recomputes basic metrics (even though they exist from 6A)
   - CELL 6C can be run without pool metrics and saved once for reuse
3. **Smart skipping**:
   - Data quality checks are skipped in 6B, 6C, and 6D (already done in 6A)
   - Pool metrics are isolated in 6D for clean restarts
4. **Resumable checkpoints**:
   - Each cell saves its output
   - If a cell crashes, you can resume from the last checkpoint

---

What Gets Recomputed vs. Added

CELL 6A: Fresh Computation

Input: `df_pipeline_02_oer_joined` (from CELL 5)

Computation: - ✅ **Fresh**: Basic cumulative metrics (cum_tb_rcvd_rtr, etc.) - ✅ **Fresh**: TB ratios (tb_ratio_rtr, tb_ratio_snr) - ✅ **Fresh**: Data quality checks (if enabled) - ❌ **Skipped**: Optional metrics (ratings given, grade, last4) - ❌ **Skipped**: Pool metrics

Output: `df_pipeline_03a_basic_metrics.feather`

CELL 6B: Recomputes Basic + Adds Optional

Input: `df_pipeline_03a_basic_metrics` (from CELL 6A)

Computation: - ⚠️ **Recomputed**: Basic cumulative metrics (redundant, but function requires it) - ⚠️ **Recomputed**: TB ratios (redundant) - ✅ **Added**: Ratings given metrics (if `include_ratings_given=True`) - ✅ **Added**: Grade-based metrics (if `include_grade_metrics=True`) - ✅ **Added**: Last4 metrics (if `include_last4_metrics=True`) - ❌ **Skipped**: Data quality checks (already done in 6A) - ❌ **Skipped**: Pool metrics

Output: `df_pipeline_03b_optional_metrics.feather`

Redundancy: ~30 minutes of redundant computation (basic metrics recomputed)

CELL 6C: Pre-Pool Prep (no pool metrics)

Input: `df_pipeline_03b_optional_metrics` (from CELL 6B) OR `df_pipeline_03a_basic_metrics` (from CELL 6A, if 6B skipped)

Computation: - ✅ **Added**: Pre-pool checkpoint (no pool metrics) - ✅ **Optional**: Precompute both TB modes (`*_fwd`, `*_bwd`) for faster experiments

Output: `df_pipeline_03c_pre_pool_metrics.feather`

CELL 6D: Adds Pool Metrics

Input: `df_pipeline_03c_pre_pool_metrics` (from CELL 6C)

Computation: - ✅ **Added**: Pool metrics (NEW - this is what 6D adds)

Output: `df_pipeline_03_oer_metrics.feather` (FINAL)

Resumable Checkpoints

How Resumability Works

Each cell follows this pattern:

```
# Check if output already exists
try:
    df_base = load_feather('df_pipeline_03a_basic_metrics')
    print_v("✅ Found existing output - skipping computation")
except FileNotFoundError:
    # Need to compute
    df_base = add_cumulative_oer_metrics(...)
    store_feather(df_base, 'df_pipeline_03a_basic_metrics')
```

Checkpoint Files

Cell	Checkpoint File	Contains
CELL 6A	`df_pipeline_03a_basic_metrics.feather`	Basic cumulative metrics only
CELL 6B	`df_pipeline_03b_optional_metrics.feather`	Basic + Optional metrics
CELL 6C	`df_pipeline_03c_pre_pool_metrics.feather`	Pre-pool checkpoint (no pool metrics)
CELL 6D	`df_pipeline_03_oer_metrics.feather`	Basic + Optional + Pool metrics (FINAL)

Fallback Logic

CELL 6B: - Tries to load `df_pipeline_03b_optional_metrics.feather` - If not found, tries `df_pipeline_03a_basic_metrics.feather` (from 6A) - If neither found, raises error (must run 6A first)

CELL 6C: - Tries to load `df_pipeline_03c_pre_pool_metrics.feather` - If not found, tries `df_pipeline_03b_optional_metrics.feather` (from 6B) - If not found, tries `df_pipeline_03a_basic_metrics.feather` (from 6A) - If none found, raises error (must run 6A first)

CELL 6D: - Tries to load `df_pipeline_03_oer_metrics.feather` - If not found, tries `df_pipeline_03c_pre_pool_metrics.feather` (from 6C) - If not found, tries `df_pipeline_03b_optional_metrics.feather` (from 6B) - If not found, tries `df_pipeline_03a_basic_metrics.feather` (from 6A) - If none found, raises error (must run 6A first)

Benefits of Resumability

1. **Crash Recovery**: If a cell crashes, you can resume from the last checkpoint
2. **Selective Re-run**: Re-run only the cells you need (e.g., just 6D if pool metrics need updating)
3. **Testing**: Test individual cells without running the full pipeline

4. **Memory Management**: Each cell cleans up Ray objects between cells

---

Performance Implications

Redundancy Cost

Scenario	Redundancy	Total Time
Run all cells (6A → 6B → 6C → 6D)	6B: ~30 min, 6C: ~min-1h, 6D: ~1-2h	~2-3 hours
Run 6A only	None	~30 minutes
Run 6A + 6D (skip 6B/6C)	6D recomputes as needed	~1-2 hours
Run 6A + 6B (skip 6C/6D)	6B: ~30 min (recomputes basic)	~1-1.5 hours

Why Redundancy Exists

The `add_cumulative_oer_metrics()` function doesn't have an "add-only" mode. It always recomputes everything it's asked to compute. The cells work around this by:

1. **Skipping expensive operations**: Data quality checks skipped in 6B, 6C, and 6D
2. **Isolated pool work**: 6C prepares pre-pool; 6D computes pool metrics only
3. **Resumable checkpoints**: You can skip cells entirely if their output exists

Optimization Strategies

Strategy 1: Skip Optional Metrics

```
CELL6_OPTIMIZE = {
    'include_ratings_given': False,  # Skip ratings given
    'include_grade_metrics': False,   # Skip grade metrics
    'include_last4_metrics': False,   # Skip last4 metrics
}
# Run: 6A only → ~30 minutes (no redundancy)
```

Strategy 2: Skip Pool Metrics

```
CELL6C = False  # Skip pre-pool
CELL6D = False  # Skip pool metrics entirely
# Run: 6A + 6B → ~1-1.5 hours (accepts ~30 min redundancy in 6B)
```

Strategy 3: Skip Data Quality Checks

```
CELL6_OPTIMIZE = {
    'check_data_quality': False,  # Skip if data already clean
}
# Saves 1-2 hours in CELL 6A
```

Best Practices

1. First-Time Run

Run all cells to get complete metrics:

```
CELL6A = True
CELL6B = True
CELL6C = True
CELL6D = True
```

Expected time: ~2-3 hours (with redundancy)

2. Subsequent Runs (If Only Basic Metrics Needed)

Run 6A only:

```
CELL6A = True
CELL6B = False
CELL6C = False
CELL6D = False
```

Expected time: ~30 minutes (no redundancy)

3. If Pool Metrics Crash

Re-run 6D only (it will resume from 6C/6B/6A):

```
CELL6A = False  # Skip (output exists)
CELL6B = False  # Skip (output exists)
CELL6C = False  # Skip pre-pool (already saved)
CELL6D = True   # Re-run pool metrics
```

Expected time: ~1-2 hours (pool metrics only)

4. Production Optimization

Disable features you don't need:

```
CELL6_OPTIMIZE = {
    'check_data_quality': False,      # Skip if data clean
    'include_ratings_given': False,    # Skip if not needed
    'include_grade_metrics': False,   # Skip if not needed
    'include_last4_metrics': False,  # Skip if not needed
    'include_pool_metrics': True,     # Keep if needed
}
CELL6B = False  # Skip 6B entirely if optional metrics not needed
CELL6C = True
CELL6D = True
```

Expected time: ~1.5-2 hours (6A + 6D, with minimal redundancy)

5. Testing New Features

Run individual cells to test:

```
# Test 6A only
CELL6A = True
CELL6B = False
CELL6C = False
CELL6D = False

# Then test 6B
CELL6A = False  # Skip (output exists)
CELL6B = True
CELL6C = False
```

Summary

Key Takeaways

1. **CELL6_OPTIMIZE provides defaults**, but cells have hardcoded overrides for optimization
2. **Each cell recomputes** what it's asked to compute (no true "add-only" mode)
3. **Redundancy exists** but is minimized by skipping expensive operations
4. **Resumable checkpoints** allow selective re-runs and crash recovery
5. **Best practice**: Run all cells first time, then skip cells you don't need

Flow Summary

```
CELL 6A: Fresh computation (basic metrics) → Save checkpoint
    ↓
CELL 6B: Recompute basic + Add optional → Save checkpoint
    ↓
CELL 6C: Pre-pool prep (no pool metrics) → Save checkpoint
    ↓
CELL 6D: Add pool metrics → Save final
```

Performance Summary

- **Fastest**: Run 6A only (~30 min)

- **Complete**: Run all cells (~2-3 hours, with redundancy)
- **Optimized**: Disable unneeded features in CELL6_OPTIMIZE

The redundancy is a trade-off for resumability and modularity. The cells are designed to be independent and resumable, which is more valuable than eliminating redundancy.

Quick Settings Reference

For specific recommended settings for common scenarios (e.g., “everything except ratings given”), see CELL6_OPTIMIZATION_RECOMMENDATIONS.md.